

Exercícios — Aula 13 — DFS em matrizes e exploração recursiva

Técnicas de Programação - 2026/02

Última atualização: October 5, 2026

Instruções

- Leia o pseudocódigo antes de executar mentalmente.
- Use apenas os quatro vizinhos ortogonais: cima, baixo, esquerda e direita.
- Acompanhe a posição atual, a matriz `visitado` e as chamadas pendentes.

Exercício 1 — Simulação de DFS em labirinto

Considere a matriz abaixo. `S` é a origem, `D` é o destino, `#` é parede e `.` é uma posição livre. A ordem de tentativa é cima, baixo, esquerda e direita.

```
S . . D
# . # .
# . . .
```

Algorithm 1: ExisteCaminho

Result: Busca o destino a partir de uma posição do labirinto.

Input : labirinto `lab`, posição (l, c) , destino (dl, dc) , matriz `visitado`

Output: boolean

```
if  $(l, c)$  está fora da matriz then
  | return false
end
if  $lab[l][c] = \#$  or  $visitado[l][c] = true$  then
  | return false
end
if  $(l, c) = (dl, dc)$  then
  | return true
end
 $visitado[l][c] \leftarrow true$ 
return ExisteCaminho( $lab, l - 1, c, dl, dc, visitado$ ) or
  ExisteCaminho( $lab, l + 1, c, dl, dc, visitado$ ) or
  ExisteCaminho( $lab, l, c - 1, dl, dc, visitado$ ) or
  ExisteCaminho( $lab, l, c + 1, dl, dc, visitado$ )
```

Complete a tabela até encontrar o destino. Registre somente as chamadas que alcançam posições livres e ainda não visitadas.

passo	chamada ativa	posição marcada	próxima tentativa	resultado
1	ExisteCaminho(0, 0)			
2				
3				
4				
5				
6				
7				

Qual condição impede que uma posição já explorada seja visitada novamente?

Exercício 2 — Quatro vizinhos e limites

Considere a posição (0, 1) na matriz abaixo.

```
. . .  
# . #  
. . .
```

Preencha a tabela. Use **sim** ou **não**.

direção	coordenada	dentro da matriz?	é parede?	pode ser explorada?
cima				
baixo				
esquerda				
direita				

Depois explique por que a verificação de limites deve acontecer antes de acessar `lab[linha][coluna]`.

Exercício 3 — Completar busca de caminho

Complete as lacunas do procedimento. O método deve devolver **true** quando alcançar o destino e **false** quando não houver caminho a partir da posição atual.

Algorithm 2: ExisteCaminho

Result: Verifica se há caminho em uma matriz.

Input : lab, (l, c) , destino (dl, dc) , visitado

Output: boolean

```
if _____ then
|   return false
end
if  $(l, c) = (dl, dc)$  then
|   return _____
end
```

$visitado[l][c] \leftarrow$ _____

return _____

Exercício 4 — Simulação de flood fill

Aplicamos `PREENCHER(tela, 1, 1, '.', 'x')` na tela abaixo.

```
. . # .  
. . # #  
# . . .
```

1. Desenhe a tela depois do preenchimento.
2. Quais células mudam de cor?
3. Por que o algoritmo deve retornar imediatamente se **original** e **nova** forem iguais?

Exercício 5 — Contar regiões livres

Uma região é formada por posições . conectadas pelas quatro direções. Complete o pseudocódigo.

Algorithm 3: ContarRegioes

Result: Conta regiões livres da matriz.

Input : matriz mapa

Output: int

$visitado \leftarrow \text{MatrizDeFalsos}(\text{Linhas}(\text{mapa}), \text{Colunas}(\text{mapa}))$

$total \leftarrow 0$

foreach posição (l, c) de mapa **do**

if _____ **then**

 MarcarRegiao($\text{mapa}, l, c, visitado$)

$total \leftarrow$ _____

end

end

return $total$

Use sua resposta para contar as regiões da matriz:

. # .

. # .

.

Exercício 6 — Casos de borda

Para cada caso, indique o resultado esperado de uma busca de caminho e justifique em uma frase.

Caso	Resultado	Justificativa
matriz vazia		
origem fora da matriz		
origem em parede		
destino em parede		
origem igual ao destino e posição livre		
matriz com uma única posição livre		